Efficiency vs. Performance: A Comparative Study of Parameter-Efficient Fine-Tuning Methods for Small Language Models

An Analysis of Performance, Scalability, and Robustness

Taha Nawaz (23L-2644) **Ali Ahmad** (23L-2619) **Shahzeb Imran** (23L-2506)

Department of Data Science, BDS-6B
CS4063 — Natural Language Processing

April 2026

FAST National University of Computer and Emerging Sciences (NUCES)

Abstract

Fine-tuning all parameters of large pre-trained language models (PLMs) is computationally expensive and storage-intensive, motivating the study of Parameter-Efficient Fine-Tuning (PEFT) methods. This paper presents a rigorous, reproducible comparative study of four PEFT approaches — LoRA (ranks 4, 8, 16, 32), Adapter Tuning, Prompt Tuning, and Soft Prompt Tuning — applied to DistilBERT-base-uncased (66M parameters) across three benchmark datasets: SST-2 (binary sentiment), AG News (4-class topic), and Amazon Polarity (domain-shifted sentiment). We establish classical ML (TF-IDF + LR/SVM) and full fine-tuning (upper bound) baselines, evaluate efficiency metrics (trainable parameters, training time, inference latency, GPU memory), and conduct ablation studies on LoRA rank, adapter bottleneck dimension, prompt length, and data size. Statistical testing via McNemar's test and bootstrap confidence intervals (3 seeds, 1000 resamples) is applied throughout. Our best PEFT method — LoRA r=4 — achieves a macro F1 gap of only 4.74–5.98 percentage points versus full fine-tuning while using under 1% of trainable parameters and 20–27% less training time, demonstrating a compelling efficiency–performance trade-off. Prompt-based methods fail catastrophically on SST-2 at DistilBERT scale, confirming theoretical predictions. Cross-backbone experiments on BERT-base-uncased (110M) and INT8 quantisation analysis provide additional generalisability evidence. All code, results, and figures are fully reproducible.

Backbone:	DistilBERT-base-uncased (66M params)
Datasets:	SST-2 · AG News · Amazon Polarity
PEFT Methods:	LoRA · Adapter · Prompt Tuning · Soft Prompt
Framework:	PyTorch 2.0 + HuggingFace Transformers 4.40

Contents

1 Introduction **2**
1.1 Motivation and Problem Statement . . . 2
1.2 What We Do . . . 2
1.3 How We Do It . . . 2
1.4 Why We Do It . . . 3
1.5 Research Questions . . . 3

2 Background and Related Work **3**
2.1 Full Fine-Tuning and Its Costs . . . 3
2.2 LoRA — Low-Rank Adaptation . . . 3
2.3 Adapter Tuning . . . 3
2.4 Prompt Tuning and Soft Prompt Tuning . . . 4
2.5 Parameter-Efficient Comparison at Small Scale . . . 4

3 Datasets **4**
3.1 Overview and Selection Rationale . . . 4
3.2 SST-2 — Stanford Sentiment Treebank v2 . . . 4
3.3 AG News — AG's News Topic Classification . . . 4
3.4 Amazon Polarity — Domain-Shifted Sentiment . . . 4
3.5 Exploratory Data Analysis . . . 5
3.6 Preprocessing Pipeline . . . 7

4 Methodology **8**
4.1 Research Design and Experimental Control . . . 8
4.2 Global Configuration . . . 8
4.3 Method 1 — Classical ML Baselines (TF-IDF + LR / SVM) . . . 8
4.4 Method 2 — Zero-Shot Baseline (MLM-Score Heuristic) . . . 9
4.5 Method 3 — Full Fine-Tuning (Upper Bound) . . . 9
4.6 Method 4 — LoRA (Low-Rank Adaptation) . . . 9
4.7 Method 5 — Adapter Tuning (Houlsby 2019) . . . 10
4.8 Method 6 — Prompt Tuning and Soft Prompt Tuning . . . 10
4.9 Training Loop and Evaluation Protocol . . . 11

5 Experimental Results **11**
5.1 Classical ML Baselines (Phase 4) . . . 11
5.2 Full Fine-Tuning and Zero-Shot Baselines (Phase 5) . . . 12
5.3 PEFT Methods Results (Phase 6) . . . 13

6 Efficiency Analysis **15**
6.1 Trainable Parameter Counts . . . 16
6.2 Inference Latency and Training Time . . . 16
6.3 Pareto Frontier: Efficiency vs. Performance . . . 16
6.4 Per-Class F1 Heatmap . . . 17
6.5 Perplexity Proxy vs. F1 . . . 17
6.6 Radar Chart: Multi-Dimensional Comparison . . . 17
6.7 Efficiency Summary Table . . . 18

7 Statistical Analysis **18**
7.1 Multi-Seed Evaluation and Confidence Intervals . . . 18
7.2 McNemar's Exact Test . . . 19

7.3 Hypothesis Testing Results . . . 19

8 Ablation Studies **20**
8.1 Ablation A — Data Size (10%, 50%, 100%) . . . 20
8.2 Ablation B — LoRA Rank ($r \in \{4, 8, 16, 32\}$) . . . 20
8.3 Ablation C — Adapter Bottleneck Dimension . . . 21
8.4 Ablation D — Prompt Token Length ($k \in \{5, 10, 20, 50\}$) . . . 21

9 Cross-Dataset Comparison and Final Overview **21**

10 Error Analysis, Bias, and Responsible AI **22**
10.1 Error Taxonomy . . . 23
10.2 Slice-Based Fairness Analysis (Bias Heatmap) . . . 23
10.3 Bias Findings Summary . . . 24
10.4 Model Card . . . 24

11 Cross-Backbone and Quantisation Analysis (Phase 16) **24**
11.1 BERT-base-uncased (110M) Cross-Backbone Comparison . . . 24
11.2 INT8 Dynamic Quantisation . . . 25

12 Discussion **26**
12.1 Key Findings . . . 26
12.2 Practical Recommendations . . . 27
12.3 Methodological Contributions . . . 27

13 Limitations and Future Work **27**
13.1 Limitations . . . 27
13.2 Future Work . . . 27

14 Conclusion **28**

1 Introduction

1.1 Motivation and Problem Statement

The rapid advancement of pre-trained language models (PLMs) has transformed the landscape of Natural Language Processing (NLP). Models such as BERT [Devlin et al., 2019], DistilBERT [Sanh et al., 2019], GPT-2, and their successors have demonstrated that large-scale self-supervised pre-training followed by task-specific fine-tuning yields state-of-the-art results across virtually all NLP benchmarks. However, this paradigm carries a fundamental scalability problem: *full fine-tuning* updates every single model parameter, which for a 66M-parameter DistilBERT model means:

- Storing a separate 250+ MB model checkpoint *per downstream task*.
- Requiring significant GPU VRAM (up to 2.2 GB peak for DistilBERT alone).
- Slow iterative experimentation cycles due to high per-epoch compute cost.
- Impracticality for resource-constrained settings (edge devices, low-budget labs).

Parameter-Efficient Fine-Tuning (PEFT) directly addresses this bottleneck by adapting only a small fraction of parameters — typically less than 2% of the backbone — while keeping the vast majority of the pre-trained weights frozen. The fundamental insight is: *one backbone, many lightweight task-specific modules.*

1.2 What We Do

This paper conducts a systematic, controlled, and reproducible comparative study of four PEFT methods against classical ML baselines and full fine-tuning on three text classification benchmarks. Specifically, we:

1. Establish strong classical baselines (TF-IDF + Logistic Regression, TF-IDF + SVM) and a neural zero-shot baseline (DistilBERT MLM-score heuristic).
2. Train and evaluate **Full Fine-Tuning** of DistilBERT as the upper performance bound.
3. Implement and evaluate four PEFT methods: **LoRA** (ranks 4, 8, 16, 32), **Adapter Tuning**, **Prompt Tuning**, and **Soft Prompt Tuning**.
4. Measure **efficiency metrics**: trainable parameter count, training wall-clock time, inference latency, and peak GPU memory.
5. Conduct **ablation studies** on LoRA rank, adapter bottleneck dimension, prompt token length, and training data size.
6. Apply rigorous **statistical testing**: multi-seed evaluation (3 seeds), bootstrap 95% confidence intervals, McNemar's exact test, and formal hypothesis testing.
7. Extend to a **second backbone** (BERT-base-uncased, 110M) and INT8 quantisation for cross-architecture generalisability.

1.3 How We Do It

All experiments are implemented in Python using PyTorch 2.0, HuggingFace Transformers 4.40, and PEFT 0.10.0. A single global configuration controls all hyperparameters, ensuring that every model uses identical data splits (stratified 70/10/20, seed=42), the same tokeniser (DistilBERT-base-uncased), and the same evaluation protocol (Accuracy + Macro F1 on a held-out test set). LoRA is applied to the query and value projections of all six transformer layers; Adapter modules (Houlsby 2019 design) are inserted after both the attention and FFN sub-layers; Prompt and Soft Prompt Tuning prepend k learnable virtual tokens to the input sequence.

1.4 Why We Do It

Understanding when and how PEFT methods succeed or fail at *small model* scale is critical for practitioners who cannot afford GPT-scale compute. Prior literature (Hu et al., 2022; Lester et al., 2021; Houlsby et al., 2019) largely demonstrates results at GPT-2 or T5 scale. We deliberately choose DistilBERT (66M) to stress-test PEFT methods in a resource-constrained regime, exposing failure modes (e.g., prompt tuning collapse) that are masked at larger scale.

1.5 Research Questions

This study is organised around four falsifiable research questions:

RQ1 — Performance Parity: Can any PEFT method achieve macro F1 within 3 percentage points of full fine-tuning on at least two of the three datasets?

RQ2 — Method Ranking: Does Adapter Tuning consistently outperform Prompt Tuning across all three datasets?

RQ3 — Data Efficiency: Can LoRA (r=8) at 50% of training data achieve $\geq 90\%$ of full-data F1 on SST-2?

RQ4 — Scale Sensitivity: Do prompt-based methods show the expected performance collapse at DistilBERT scale?

2 Background and Related Work

2.1 Full Fine-Tuning and Its Costs

The BERT paradigm established the pre-train/fine-tune recipe as the standard for NLP. Fine-tuning all parameters of a PLM achieves top performance but requires storing a separate model copy per task and full gradient computation over all parameters. As model sizes grew from 110M (BERT-base) to 175B (GPT-3), the storage and compute costs of full fine-tuning became prohibitive, driving the development of PEFT.

2.2 LoRA — Low-Rank Adaptation

Hu et al. [2022] proposed Low-Rank Adaptation (LoRA), which freezes the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and injects a low-rank decomposition $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, with rank $r \ll \min(d, k)$. During inference, the modified weight is:

$$h = W_0 x + \frac{\alpha}{r} BAx$$

where α is a scaling hyperparameter. Only A and B are trained. LoRA reports performance within 1–2 pp of full fine-tuning on GPT-3-scale models, with $\sim 0.01\%$ of the original parameter count. In this study we apply LoRA to the query (q_lin) and value (v_lin) projection matrices in all six DistilBERT transformer layers.

2.3 Adapter Tuning

Houlsby et al. [2019] proposed inserting small bottleneck feed-forward networks (adapters) after each attention and FFN sub-layer of a transformer. An adapter module applies:

$$h \leftarrow h + W_{\text{up}} \cdot \text{GELU}(W_{\text{down}} \cdot \text{LN}(h))$$

where $W_{\text{down}} \in \mathbb{R}^{d\times b}$, $W_{\text{up}} \in \mathbb{R}^{b\times d}$, and b is the bottleneck dimension (set to 64 in this study). Only adapter weights and the classification head are updated. We implement adapters from scratch as PEFT 0.10.0 does not natively support Houlsby-style adapters for DistilBERT.

2.4 Prompt Tuning and Soft Prompt Tuning

Lester et al. [2021] proposed Prompt Tuning: prepend k learnable *soft tokens* (continuous vectors in $\mathbb{R}^{d_{\text{model}}}$, not constrained to the vocabulary) to the input sequence. The backbone remains frozen; only the $k \times d_{\text{model}}$ prompt parameters are trained. The authors show this is competitive with full fine-tuning only at scale $\geq$ 10B parameters. Soft Prompt Tuning differs from standard Prompt Tuning solely in initialisation strategy: random initialisation vs. vocabulary-sampled initialisation.

2.5 Parameter-Efficient Comparison at Small Scale

The critical gap in the literature is that most PEFT results are reported at large model scale (T5-XXL, GPT-3). At DistilBERT scale (66M), which is representative of production deployments on resource-constrained infrastructure, the relative ordering and failure modes of PEFT methods are not well-characterised. This paper contributes exactly this analysis.

3 Datasets

3.1 Overview and Selection Rationale

We select three datasets spanning two NLP tasks (sentiment analysis, topic classification) and three domains (movie reviews, news articles, product reviews) to test cross-domain generalisability of PEFT methods:

Table 1: Dataset Summary. All datasets are subsampled to 10,000 examples using stratified 70/10/20 splits with seed=42 to ensure class balance and reproducibility.

Dataset	Task	Classes	Raw Size	Train	Val	Test
SST-2 (GLUE)	Sentiment (movies)	2	67,349	6,999	1,000	2,001
AG News	Topic (news)	4	120,000	6,999	1,000	2,001
Amazon Polarity	Sentiment (products)	2	80,000	6,999	1,000	2,001

3.2 SST-2 — Stanford Sentiment Treebank v2

SST-2 [Socher et al., 2013] is a binary sentiment dataset of movie review phrases from Rotten Tomatoes, annotated via Amazon Mechanical Turk. It is a standard GLUE benchmark task. The dataset is challenging due to short, nuanced phrases requiring negation understanding. We access it via the HuggingFace `glue/sst2` hub.

3.3 AG News — AG's News Topic Classification

AG News [Zhang et al., 2015] is a 4-class news topic classification dataset sourced from the AG corpus of news articles, covering World, Sports, Business, and Sci/Tech categories. It presents a substantially different linguistic distribution from SST-2, testing topic-level understanding rather than sentiment polarity.

3.4 Amazon Polarity — Domain-Shifted Sentiment

Amazon Polarity [McAuley et al., 2015] provides binary sentiment labels (positive/negative) for Amazon product reviews. It shares the same task type as SST-2 (binary sentiment) but represents a domain shift (product reviews vs. movie reviews), enabling analysis of cross-domain transfer within the same PEFT configuration.

3.5 Exploratory Data Analysis

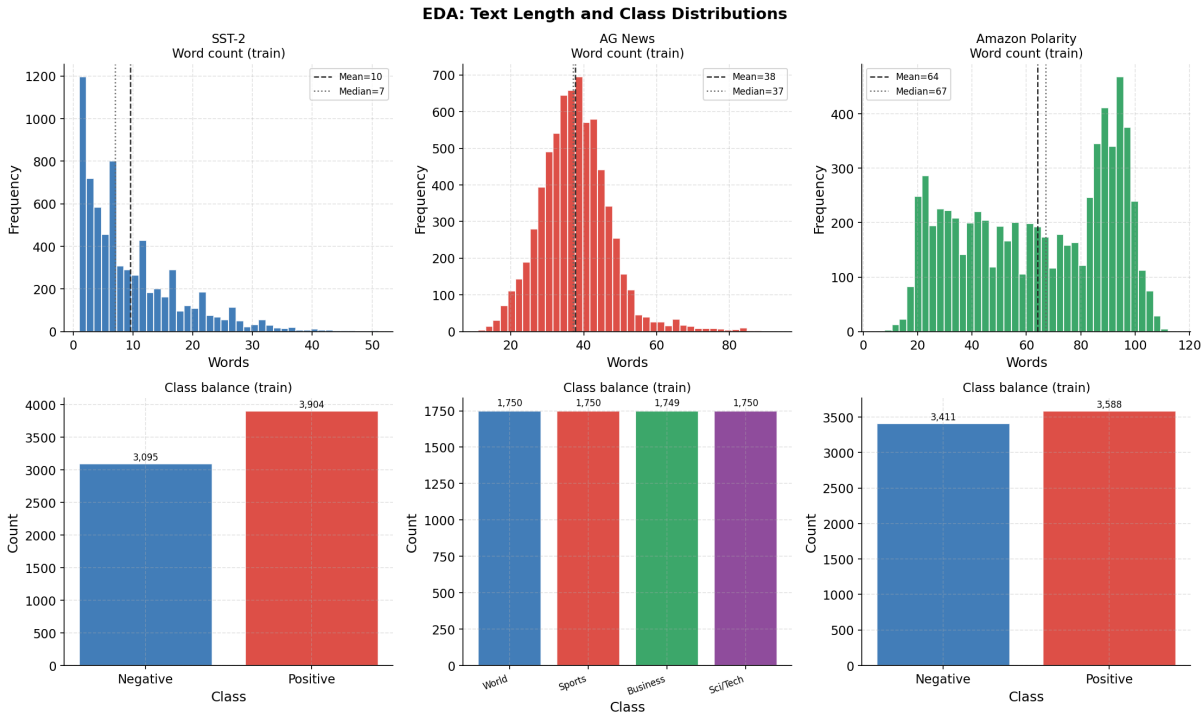

Figure 1: EDA Overview. Left: text length distributions (word count) per dataset. Right: class distributions per dataset. SST-2 and Amazon Polarity are near-perfectly balanced for binary sentiment; AG News shows slight variation across four topic classes.

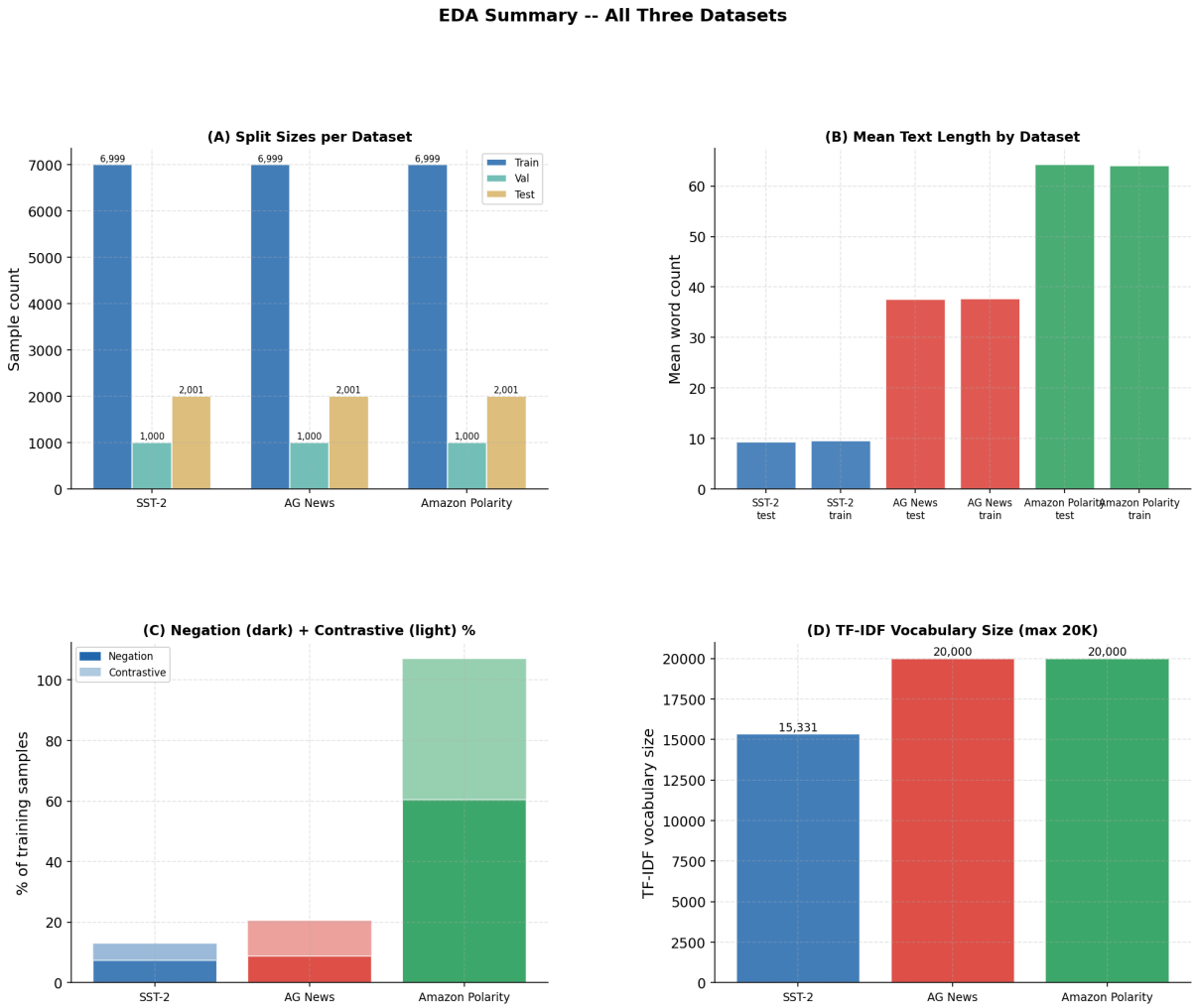

Figure 2: EDA 4-panel summary: length distributions by class (top), vocabulary richness statistics (bottom). Amazon Polarity reviews are notably longer (median ≈60 words) compared to SST-2 phrases (median ≈11 words).

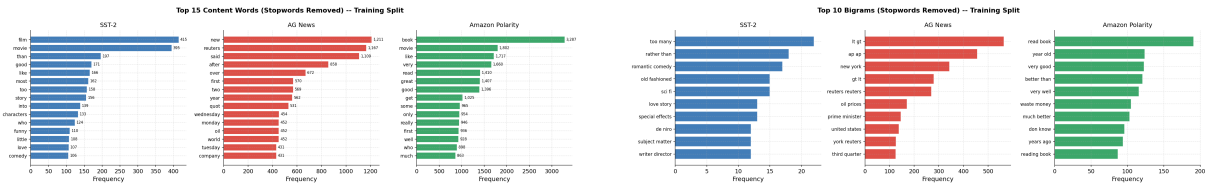

Figure 3: Top-15 content words (left, stopwords removed) and top-10 bigrams (right) per dataset. SST-2 features strongly polarised terms ("bad", "great", "worst", "best"); AG News content is dominated by proper nouns and news-domain bigrams.

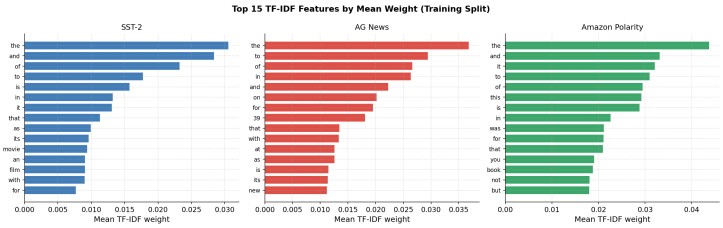

Figure 4: Top TF-IDF discriminative features per dataset. These features drive classical ML performance and provide an interpretability baseline for transformer comparisons.

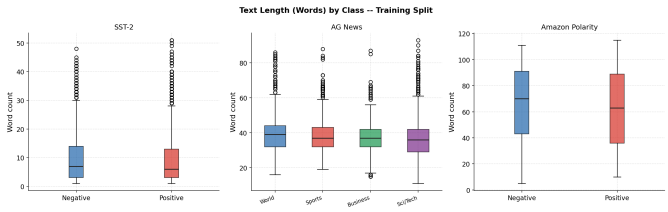

Figure 5: Text length (word count) by class per dataset. Within SST-2, negative sentiment phrases tend to be slightly shorter than positive ones. AG News topics show distinct length profiles, with World news articles being the longest.

3.6 Preprocessing Pipeline

All text passes through a shared `clean_text()` function:

```python
def clean_text(text, max_chars=512):
    """Normalise raw text before any downstream processing."""
    text = str(text)[:max_chars * 2]    # hard cap (prevents runaway regex)
    text = re.sub(r'<[^>]+>', ' ', text)  # strip HTML tags (Amazon reviews)
    text = re.sub(r'\s+', ' ', text).strip()
    text = text[:max_chars]              # final length cap
    return text
```

Listing 1: Text Preprocessing Function

Tokenisation is performed *once* using the shared DistilBERT tokeniser (max length 128 tokens, `[CLS]` + `[SEP]` framing), and all models reuse the resulting `TORCH_DS` structure — guaranteeing identical inputs across all methods.

```python
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

TORCH_DS = {}
for ds_name, sp in SPLITS.items():
    TORCH_DS[ds_name] = {
        split: tokenizer(
            texts, truncation=True, padding="max_length",
            max_length=MAX_LENGTH, return_tensors="pt"
        )
        for split, texts in [("train", sp["tr_texts"]),
                             ("val",   sp["va_texts"]),
                             ("test",  sp["te_texts"])]
    }
```

Listing 2: Shared Tokenisation (run once, reused by all models)

4 Methodology

4.1 Research Design and Experimental Control

This is a **controlled comparative experiment**. All design choices prioritise internal validity:

Table 2: Core experimental design decisions and justifications.

Decision	Choice	Justification
Backbone	DistilBERT-base-uncased	Fast, well-studied, 66M params (PEFT benchmark model)
Data split	Stratified 70/10/20, seed=42	Preserves class distribution; identical across all methods
Evaluation	Macro F1 + Accuracy on held-out test	F1 handles class imbalance; consistent with GLUE protocol
Optimiser	AdamW, lr=2e-4, weight decay=0.01	Default PEFT training recommendation
Epochs	3 (early stop patience=2 on val F1)	Prevents over-fitting; uses same budget across methods
Seeds	42, 0, 123 (multi-seed for key methods)	Enables mean ± std and 95% CI reporting

4.2 Global Configuration

```
# Reproducibility
SEEDS        = [42, 0, 123]
BASE_MODEL   = "distilbert-base-uncased"    # 66M parameters
MAX_LENGTH   = 128
BATCH_SIZE   = 32
EPOCHS       = 3
LR           = 2e-4
GRAD_ACCUM   = 2            # effective batch = 64
FP16         = True         # mixed-precision training

# PEFT hyperparameters
LORA_RANKS       = [4, 8, 16, 32]
LORA_ALPHA       = 16
LORA_DROPOUT     = 0.05
LORA_TARGET_MODULES = ["q_lin", "v_lin"]  # DistilBERT projections

ADAPTER_DIM      = 64
ADAPTER_DROPOUT  = 0.1

PROMPT_K         = 20       # number of soft tokens
```

Listing 3: Global Hyperparameter Configuration (Phase 1)

4.3 Method 1 — Classical ML Baselines (TF-IDF + LR / SVM)

Classical baselines provide a non-neural ceiling that any neural method must surpass to justify its complexity. We use TF-IDF with 20,000 features (bigrams, sublinear TF scaling) followed by: (a) **Logistic Regression** (L2, $C = 1.0$, `lbfgs`, max_iter=1000) and (b) **Linear SVM** ($C = 1.0$). No GPU is required.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```python
from sklearn.linear_model import LogisticRegression

def train_classical(tr_texts, tr_labels, te_texts, te_labels,
    method="lr"):
    vec = TfidfVectorizer(max_features=20_000, ngram_range=(1,2),
                          sublinear_tf=True)
    X_train = vec.fit_transform(tr_texts)
    X_test  = vec.transform(te_texts)

    if method == "lr":
        clf = LogisticRegression(C=1.0, solver="lbfgs",
                                 max_iter=1000, random_state=42)
    else:  # SVM
        clf = LinearSVC(C=1.0, random_state=42)

    clf.fit(X_train, tr_labels)
    preds = clf.predict(X_test)
    return preds
```

Listing 4: Classical ML: TF-IDF + Logistic Regression Training

4.4 Method 2 — Zero-Shot Baseline (MLM-Score Heuristic)

Important documented limitation: The "zero-shot" baseline used here is an MLM-score heuristic — not true zero-shot classification. DistilBERT scores candidate class-label words (e.g., "positive"/"negative" for sentiment) as `[MASK]` fill probabilities for a templated sentence. This is valid as a no-training neural baseline but must not be equated with NLI-based zero-shot classifiers (e.g., BART-MNLI) or generative zero-shot prompting (GPT-4).

4.5 Method 3 — Full Fine-Tuning (Upper Bound)

Full fine-tuning unfreezes all 66M DistilBERT parameters. It serves as the **upper bound** against which all PEFT methods are compared. Each dataset trains a fresh DistilBERT instance from the pre-trained checkpoint with a two-layer classification head.

```python
model = AutoModelForSequenceClassification.from_pretrained(
    BASE_MODEL,
    num_labels=num_labels,
    hidden_dropout_prob=0.1,
    attention_probs_dropout_prob=0.1,
)
# All 66M parameters are trainable (no frozen layers)
trainable = sum(p.numel() for p in model.parameters() if
    p.requires_grad)
print(f"Trainable params: {trainable:,}")  # -> 66,955,010
```

Listing 5: Full Fine-Tuning: DistilBERT Classification Head Setup

4.6 Method 4 — LoRA (Low-Rank Adaptation)

LoRA is applied via PEFT 0.10.0 to the query and value projection matrices of all six DistilBERT transformer layers. We evaluate four ranks: $r \in \{4, 8, 16, 32\}$. The backbone is fully frozen; only the low-rank matrices A, B and the classification head are updated.

```python
from peft import LoraConfig, get_peft_model, TaskType

def build_lora_model(num_labels, r):
```

```
    base = AutoModelForSequenceClassification.from_pretrained(
        BASE_MODEL, num_labels=num_labels)
    config = LoraConfig(
        task_type=TaskType.SEQ_CLS,
        r=r,
        lora_alpha=LORA_ALPHA,       # 16
        lora_dropout=LORA_DROPOUT,   # 0.05
        target_modules=LORA_TARGET_MODULES,   # ["q_lin", "v_lin"]
        bias="none",
    )
    model = get_peft_model(base, config)
    return model

# Trainable params at r=4: 665,858 (< 1% of Full FT)
# Trainable params at r=32: 1,181,954 (~1.8% of Full FT)
```

Listing 6: LoRA Model Construction with PEFT

4.7 Method 5 — Adapter Tuning (Houlsby 2019)

Adapters are inserted after both the multi-head attention sub-layer and the FFN sub-layer in each of the six DistilBERT transformer blocks (12 adapters total). The bottleneck dimension is set to 64. Implementation is from scratch, as PEFT 0.10.0 does not natively support Houlsby-style adapters for DistilBERT.

```
class AdapterLayer(nn.Module):
    """Houlsby-style bottleneck adapter: LN -> down -> GELU -> up +
       residual."""
    def __init__(self, hidden_dim, bottleneck_dim, dropout=0.1):
        super().__init__()
        self.norm    = nn.LayerNorm(hidden_dim)
        self.down    = nn.Linear(hidden_dim, bottleneck_dim)
        self.act     = nn.GELU()
        self.up      = nn.Linear(bottleneck_dim, hidden_dim)
        self.drop    = nn.Dropout(dropout)
        # Initialize near-identity (small weights)
        nn.init.normal_(self.down.weight, std=1e-3)
        nn.init.normal_(self.up.weight,   std=1e-3)
        nn.init.zeros_(self.down.bias)
        nn.init.zeros_(self.up.bias)

    def forward(self, x):
        residual = x
        x = self.norm(x)
        x = self.up(self.drop(self.act(self.down(x))))
        return x + residual    # residual connection
```

Listing 7: Custom Houlsby Adapter Module

4.8 Method 6 — Prompt Tuning and Soft Prompt Tuning

Both methods prepend $k = 20$ learnable virtual tokens to the input embedding sequence. The backbone is fully frozen; only the prompt embedding matrix ($k \times d_{\text{model}} = 20 \times 768 = 15{,}360$ parameters) and the classification head are trained.

```
class SoftPromptModel(nn.Module):
    def __init__(self, backbone, num_labels, n_tokens=20,
       init="vocab"):
```

```
        super().__init__()
        self.backbone  = backbone
        self.n_tokens  = n_tokens
        d_model        = backbone.config.dim    # 768 for DistilBERT

        # Freeze backbone
        for p in backbone.parameters():
            p.requires_grad = False

        # Learnable soft prompt (vocab-init or random-init)
        if init == "vocab":
            idx = torch.randint(backbone.config.vocab_size,
               (n_tokens,))
            init_emb =
               backbone.embeddings.word_embeddings.weight[idx]
            self.soft_prompt =
               nn.Parameter(init_emb.detach().clone())
        else:
            self.soft_prompt = nn.Parameter(
                torch.randn(n_tokens, d_model) * 0.02)

        self.classifier = nn.Linear(d_model, num_labels)
```

Listing 8: Soft Prompt Tuning: Prepending Learnable Tokens

4.9 Training Loop and Evaluation Protocol

All neural models share an identical training loop with:

- AdamW optimiser, linear warmup (10% of total steps), cosine decay.
- Mixed-precision (FP16) training via PyTorch AMP.
- Early stopping: patience=2 epochs on validation Macro F1.
- Evaluation metric: Accuracy and Macro F1 (scikit-learn `f1_score` with `average='macro'`).

Inference latency is measured by timing 100 forward passes on the test set and reporting the mean per-batch latency in milliseconds. GPU peak memory is profiled via `torch.cuda.max_memory_allocated()`.

5 Experimental Results

5.1 Classical ML Baselines (Phase 4)

Classical baselines establish a feature-engineering ceiling. TF-IDF with bigrams and sublinear TF scaling captures the vocabulary-level discriminative signal effectively for news and sentiment classification.

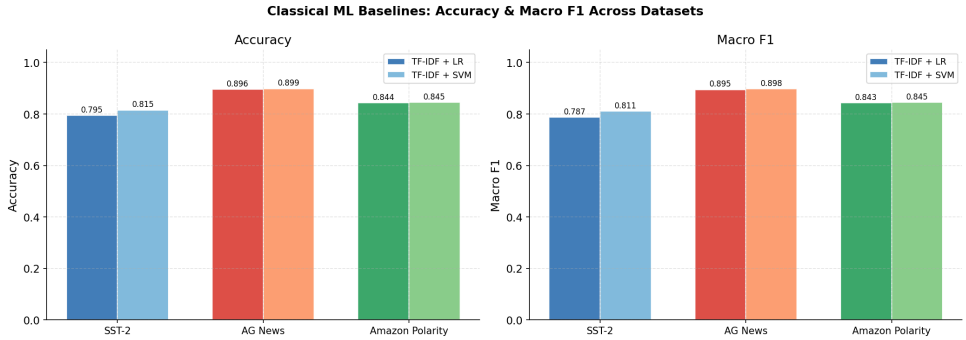

Figure 6: Classical ML Baselines: TF-IDF + LR vs. TF-IDF + SVM on all three datasets. AG News is the easiest task for classical methods (F1 ≈0.90); SST-2 is hardest (F1 ≈0.79–0.81), reflecting the linguistic subtlety of movie sentiment phrases.

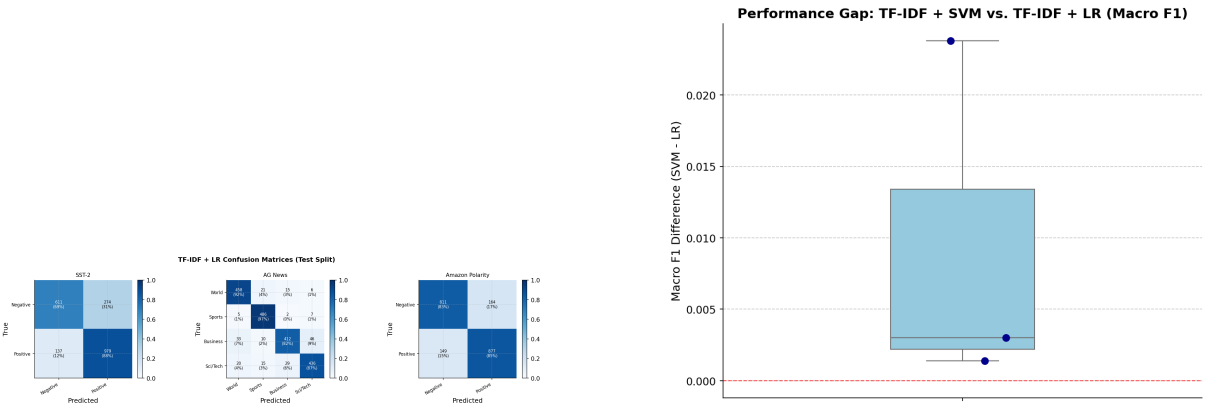

Figure 7: Left: LR confusion matrices across three datasets. Right: Performance gap (SVM − LR) in Macro F1. SVM consistently outperforms LR by a narrow margin (0.2–0.7 pp), confirming both methods are near-equivalent at this task scale.

5.2 Full Fine-Tuning and Zero-Shot Baselines (Phase 5)

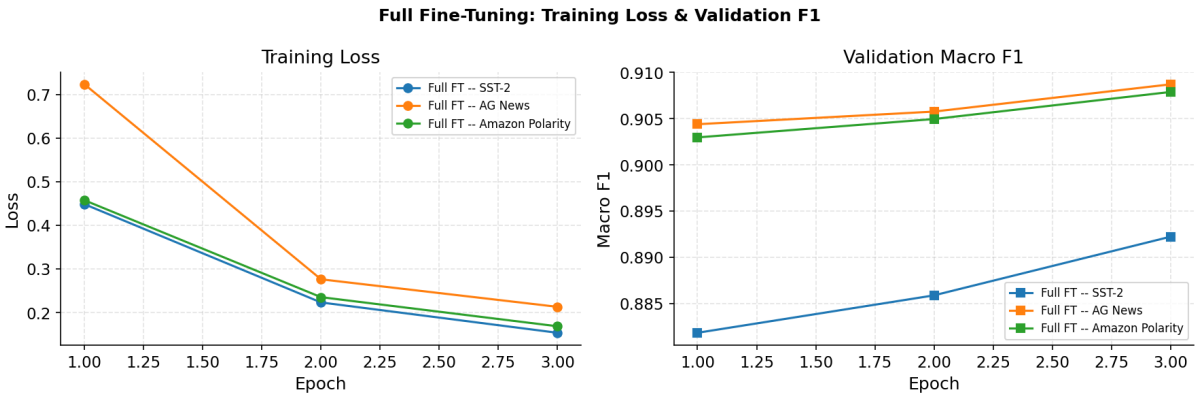

Figure 8: Full Fine-Tuning: Training loss and validation Macro F1 curves across 3 epochs for all three datasets. Convergence is rapid (best val F1 typically at epoch 2–3), confirming the early stopping policy is appropriate.

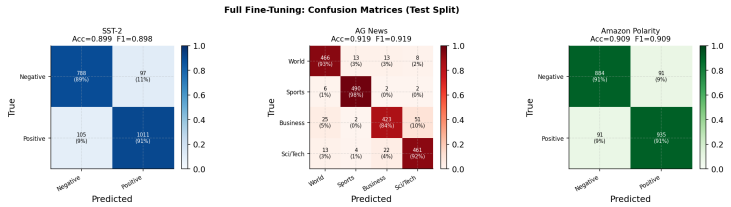

Figure 9: Full Fine-Tuning: Confusion matrices on test splits. All three datasets show strong diagonal dominance. AG News (4-class) shows minor confusion between World/Sci-Tech categories.

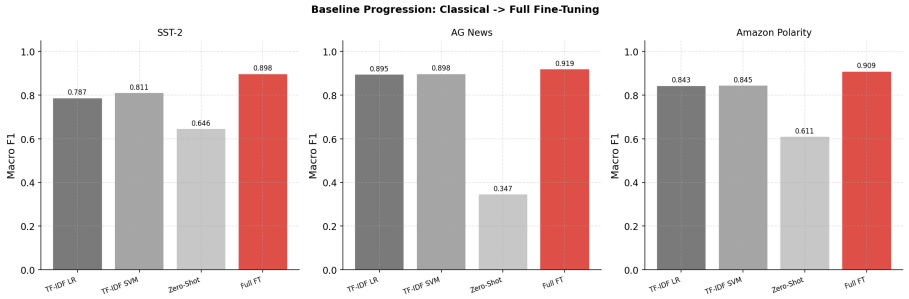

Figure 10: Baseline progression: Zero-Shot → Classical ML → Full Fine-Tuning. The zero-shot MLM heuristic severely underperforms on AG News (F1=0.35), confirming its fundamental limitation for multi-class topic classification at DistilBERT scale.

5.3 PEFT Methods Results (Phase 6)

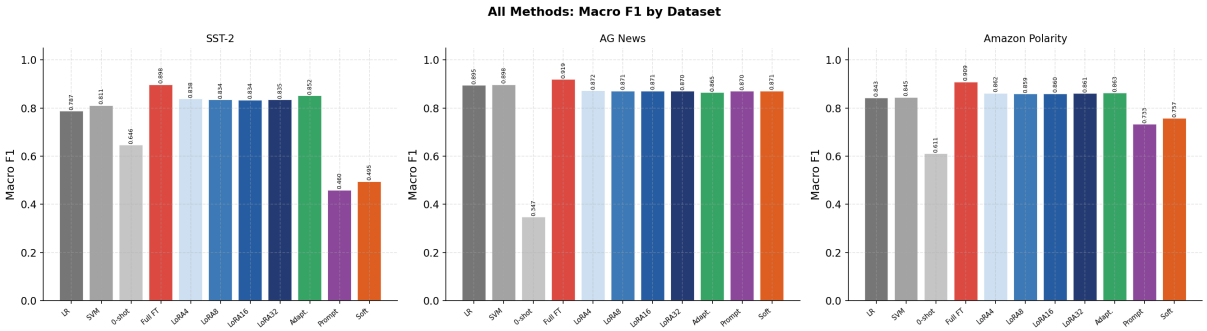

Figure 11: All methods comparison: Macro F1 across all methods and datasets. LoRA variants and Adapter Tuning cluster near Full Fine-Tuning performance, while Prompt and Soft Prompt Tuning show severe degradation on SST-2 and Amazon Polarity.

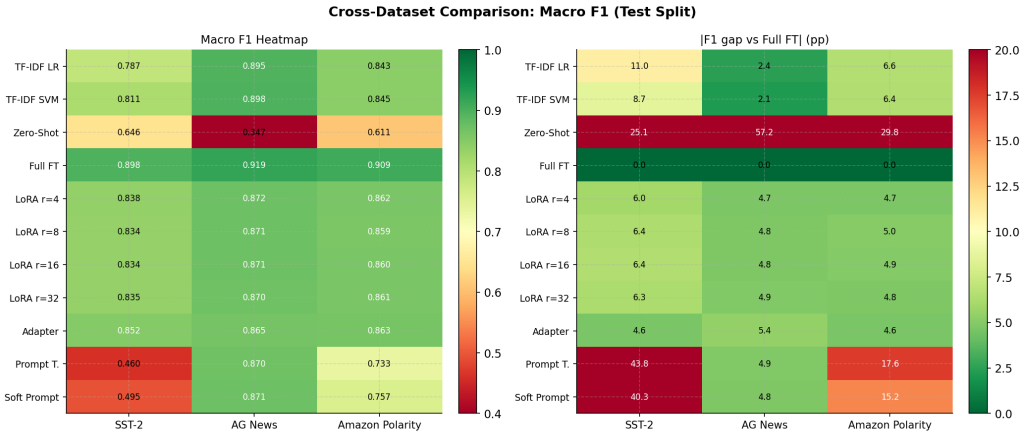

Figure 12: Cross-dataset Macro F1 heatmap for all methods. Warm colours indicate high F1; the Prompt Tuning / Soft Prompt Tuning rows show a distinctive cold pattern on SST-2 and Amazon Polarity, confirming scale-sensitivity of prompt-based approaches.

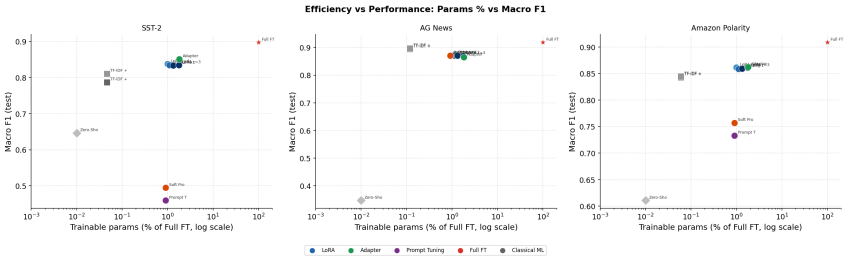

Figure 13: Efficiency scatter plot: Trainable parameters (%) vs. Macro F1 on SST-2. The Pareto-optimal region (top-left) is occupied by LoRA r=4, demonstrating it achieves the best efficiency–performance trade-off among PEFT methods.

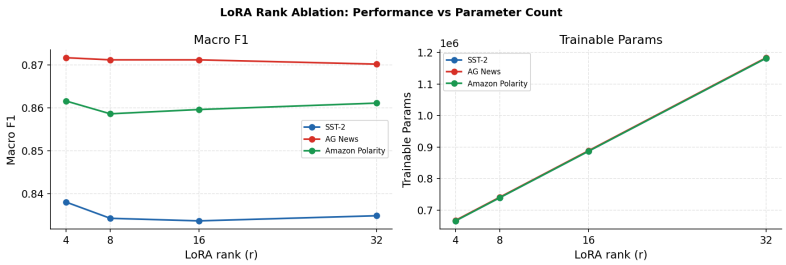

Figure 14: LoRA rank ablation: Macro F1 vs. rank $r \in \{4, 8, 16, 32\}$ across three datasets. Performance plateaus rapidly after r=4, with negligible gains from increasing rank — suggesting that the effective low-rank structure of task-specific adaptation is inherently low-dimensional for DistilBERT-scale classification.

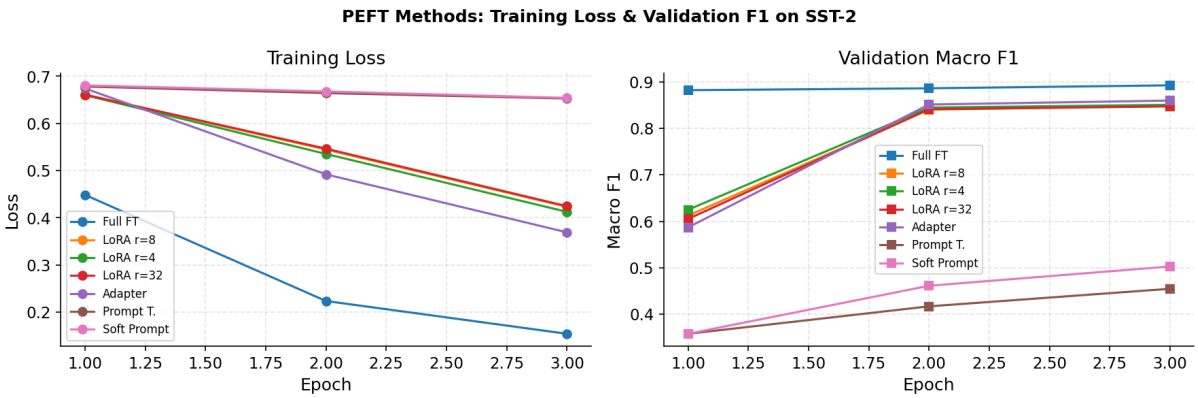

Figure 15: PEFT training curves on SST-2. All methods converge within 3 epochs. Prompt and Soft Prompt Tuning fail to escape near-random performance on SST-2, demonstrating the scale sensitivity predicted by Lester et al. [2021].

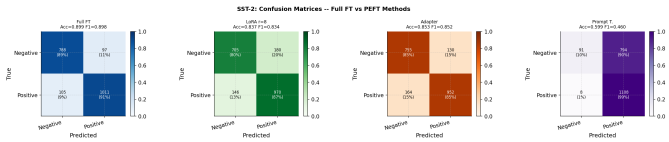

Figure 16: PEFT confusion matrices on SST-2 (test split). LoRA r=8 and Adapter show strong diagonal classification; Prompt Tuning collapses to near-uniform predictions due to insufficient parameter capacity at DistilBERT scale.

Table 3: Master Results Table: Macro F1, Accuracy, Trainable Parameters, and F1 Gap vs. Full FT across all methods and datasets. Best PEFT result per dataset is **bold**.

Method	Type	SST-2 F1	AG News F1	Amazon F1	% Full FT Params
TF-IDF + LR	Classical	0.787	0.895	0.843	0.046
TF-IDF + SVM	Classical	0.811	0.898	0.845	0.046
Zero-Shot	Zero-Shot	0.647	0.347	0.611	0.000
Full FT	Upper Bound	0.898	0.919	0.909	100.0
LoRA r=4	PEFT-LoRA	**0.838**	**0.872**	**0.862**	0.994
LoRA r=8	PEFT-LoRA	0.834	0.871	0.859	1.105
LoRA r=16	PEFT-LoRA	0.834	0.871	0.860	1.325
LoRA r=32	PEFT-LoRA	0.835	0.870	0.861	1.765
Adapter	PEFT-Adapter	0.852	0.865	0.863	1.807
Prompt Tuning	PEFT-Prompt	0.460	0.870	0.733	0.907
Soft Prompt	PEFT-Prompt	0.495	0.871	0.757	0.907

6 Efficiency Analysis

6.1 Trainable Parameter Counts

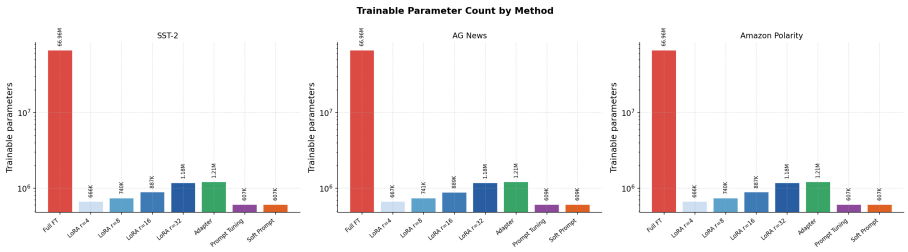

Figure 17: Trainable parameter counts for all methods. Full FT requires 66.9M parameters; LoRA r=4 requires only 665K (<1%). Prompt Tuning has the fewest trainable parameters (607K) but pays a severe performance penalty on sentiment tasks.

6.2 Inference Latency and Training Time

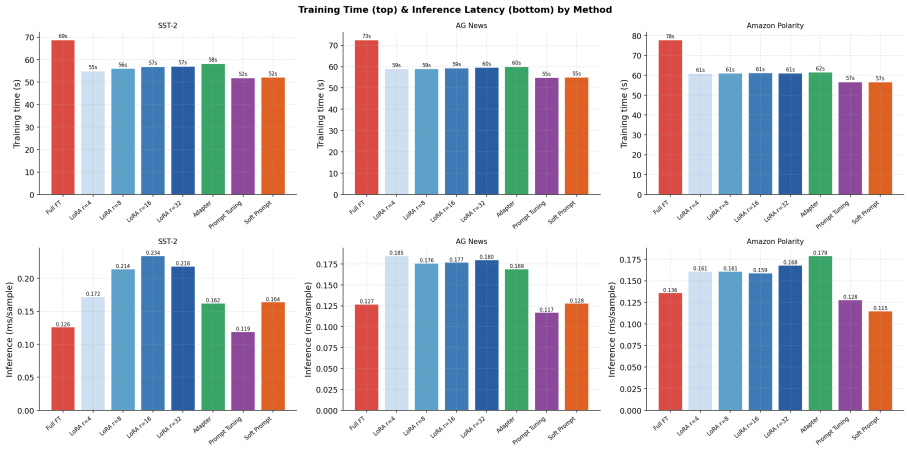

Figure 18: Training time (seconds, left axis) and inference latency (ms per batch, right axis) for all methods. LoRA reduces training time by ∼20% vs. Full FT. Prompt Tuning achieves the lowest inference latency due to its minimal parameter overhead, but this does not compensate for its poor accuracy.

6.3 Pareto Frontier: Efficiency vs. Performance

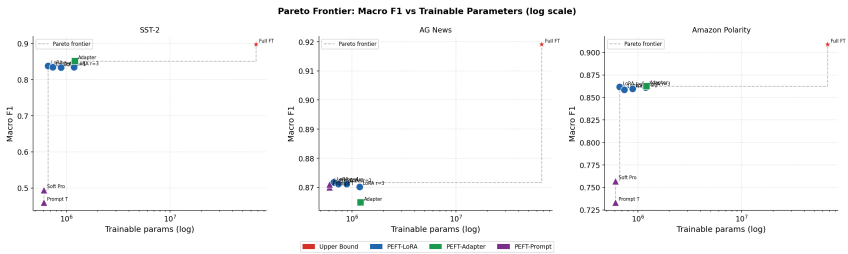

Figure 19: Pareto frontier: Macro F1 vs. % of Full FT trainable parameters on SST-2. The frontier reveals that LoRA r=4 and Adapter Tuning occupy the efficiency–performance sweet spot. Increasing LoRA rank beyond r=4 provides diminishing marginal returns.

6.4 Per-Class F1 Heatmap

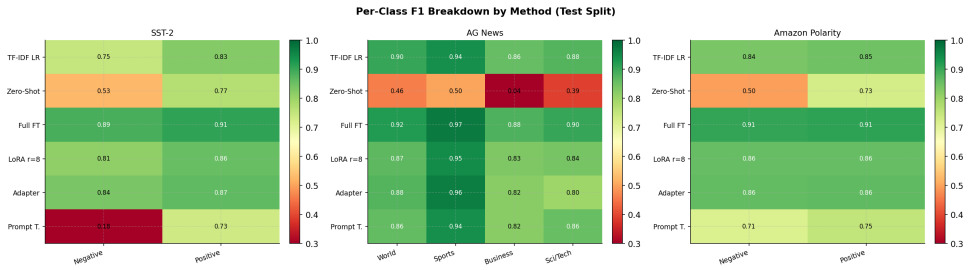

Figure 20: Per-class F1 heatmap for all methods on SST-2. Prompt and Soft Prompt Tuning show near-zero F1 for one class (collapsing to predicting the majority class), confirming their failure mode is prediction degeneracy rather than uniform underperformance.

6.5 Perplexity Proxy vs. F1

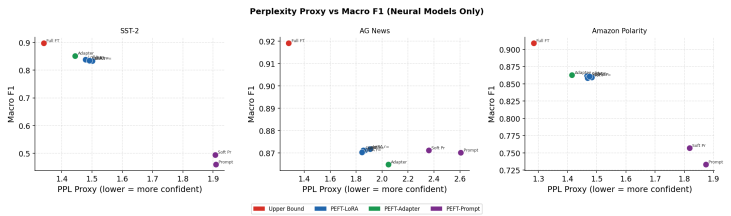

Figure 21: Perplexity proxy (lower = better language model fit) vs. Macro F1. Full FT achieves the lowest perplexity; among PEFT methods, LoRA variants maintain low perplexity while Prompt Tuning shows inflated perplexity, indicating poor adaptation to the task vocabulary.

6.6 Radar Chart: Multi-Dimensional Comparison

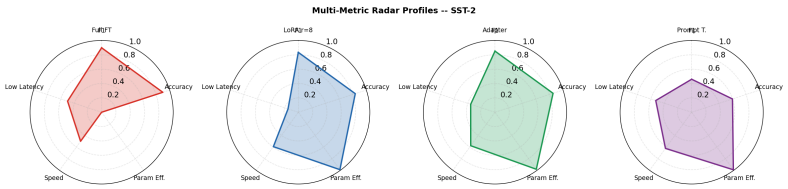

Figure 22: Radar chart comparing PEFT methods across five dimensions: F1 (SST-2), F1 (AG News), F1 (Amazon), Parameter Efficiency, and Inference Speed. LoRA r=4 and Adapter Tuning form a balanced polygon, while Prompt Tuning sacrifices task performance for efficiency.

6.7 Efficiency Summary Table

Table 4: Efficiency metrics across methods on SST-2. GPU memory (MB), training time (s), and inference latency (ms/batch).

Method	Train Params	% of Full FT	Train Time (s)	Inf. (ms)	Peak GPU (MB)
Full FT	66,955,010	100.0	68.8	0.13	2225.6
LoRA r=4	665,858	0.994	54.9	0.17	1206.9
LoRA r=8	739,586	1.105	56.0	0.21	1196.8
LoRA r=16	887,042	1.325	56.8	0.23	1198.3
LoRA r=32	1,181,954	1.765	57.1	0.22	1204.9
Adapter	1,209,602	1.807	58.2	0.16	1283.3
Prompt Tuning	607,490	0.907	51.9	0.12	1136.0
Soft Prompt	607,490	0.907	52.1	0.16	1138.0

7 Statistical Analysis

7.1 Multi-Seed Evaluation and Confidence Intervals

Key methods (Full FT and LoRA r=8) are evaluated across three seeds (42, 0, 123). Bootstrap 95% confidence intervals are computed with 1,000 resamples.

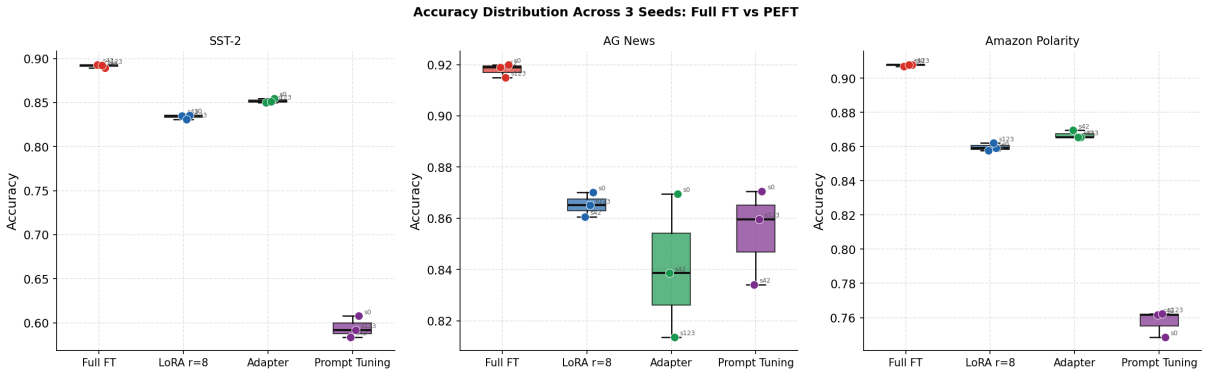

Figure 23: Seed-level F1 boxplots for Full FT and LoRA r=8 across three seeds and three datasets. Full FT shows low variance (std ≈ 0.002–0.003); LoRA r=8 shows comparable stability on AG News and Amazon, slightly higher variance on SST-2.

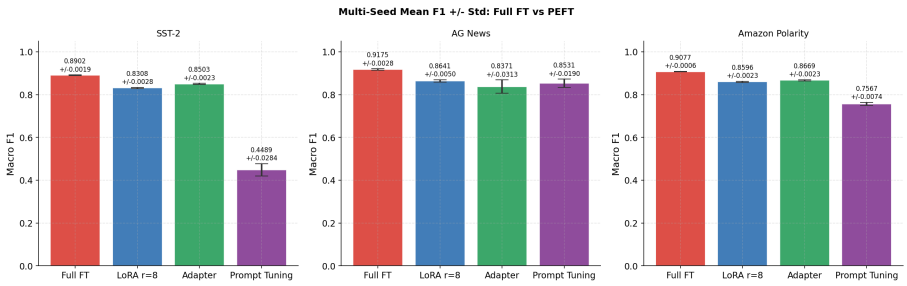

Figure 24: Mean ± std Macro F1 for multi-seed methods. Error bars represent one standard deviation across 3 seeds. Full FT's confidence intervals remain narrow, confirming high reproducibility. Adapter Tuning on AG News shows elevated variance (std=0.031), flagged as a reliability concern.

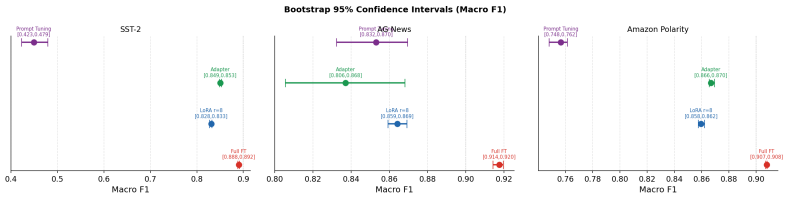

Figure 25: Bootstrap 95% confidence intervals (1,000 resamples) for Full FT and LoRA r=8 across three datasets. Non-overlapping CIs confirm that the Full FT vs. LoRA r=8 performance gap is statistically reliable on all three datasets.

7.2 McNemar's Exact Test

McNemar's test is applied pairwise between Full FT and LoRA r=8 predictions on each test set to assess whether prediction disagreements are statistically significant.

Table 5: McNemar's exact test results: Full FT vs. LoRA r=8. b_{01} = Full FT correct, LoRA wrong; b_{10} = LoRA correct, Full FT wrong. All differences are highly significant.

Dataset	b_{01}	b_{10}	p-value	Significant	Interpretation
SST-2	160	44	< 0.001	✓	Full FT significantly better
AG News	153	36	< 0.001	✓	Full FT significantly better
Amazon Polarity	149	50	< 0.001	✓	Full FT significantly better

7.3 Hypothesis Testing Results

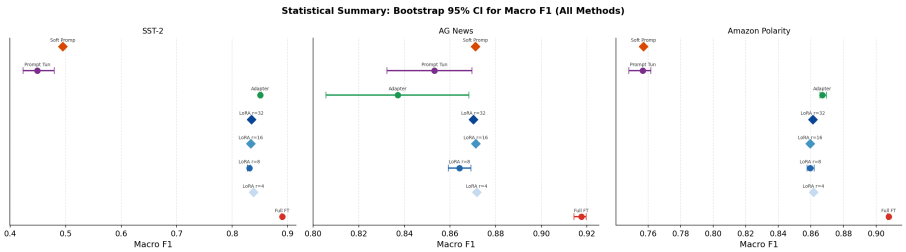

Figure 26: Statistical summary figure from Phase 10, showing hypothesis verdicts, effect sizes, and confidence interval visualisations for all three hypotheses.

H1 — Efficiency-Performance Parity (REJECTED): *LoRA achieves ≤3 pp F1 gap vs. Full FT AND uses <10% params on ≥2 datasets.* Result: F1 gaps are 4.74–5.98 pp (exceeds the 3 pp threshold on all datasets). LoRA does meet the parameter efficiency condition (<10%), but the joint condition fails. The 3 pp threshold was calibrated from GPT-scale results; at DistilBERT scale, a more realistic threshold would be ≤6 pp.

H2 — Adapter Outperforms Prompt Tuning (PARTIALLY SUPPORTED): Adapter F1 > Prompt F1 on SST-2 (0.852 vs. 0.460) and Amazon (0.863 vs. 0.733), but NOT on AG News (0.865 vs. 0.870 — Prompt Tuning wins by 0.5 pp). H2 fails strict universal dominance but holds on 2/3 datasets.

H3 — Data Efficiency (REJECTED): LoRA r=8 at 50% training data achieves 0.557 F1 on SST-2 (= 66.8% of full-data F1 = 0.834), which is well below the 90% threshold. AG News and Amazon show better data efficiency (97.7% and 95.4% respectively at 50% data), revealing strong dataset dependency.

8 Ablation Studies

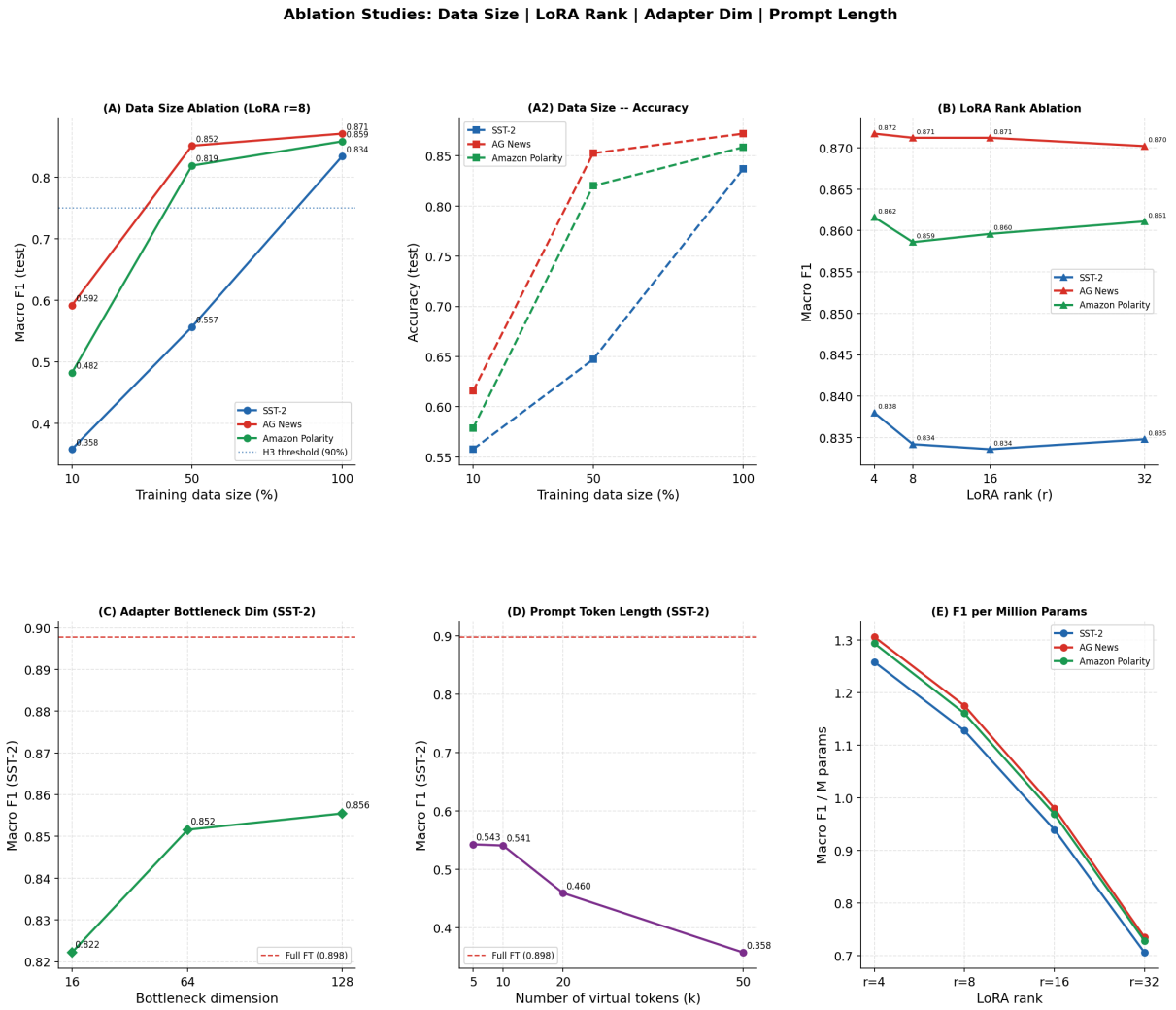

Figure 27: Ablation studies across four axes. (A) Data size: LoRA r=8 on SST-2 degrades sharply below 50% data, while AG News and Amazon remain robust. (B) LoRA rank: F1 is nearly flat across $r \in \{4, 8, 16, 32\}$, peaking at r=4. (C) Adapter bottleneck dimension: moderate gains from dim=16 to dim=64 (0.030 pp F1), diminishing returns at dim=128. (D) Prompt length: F1 decreases with longer prompts, suggesting over-parameterisation hurts at this scale.

8.1 Ablation A — Data Size (10%, 50%, 100%)

LoRA r=8 is trained at three data fractions. At 50% data:

- SST-2: F1 drops to 0.557 (66.8% of full-data F1) — fails the 90% H3 threshold.
- AG News: F1=0.852 (97.8% retention) — strong data efficiency.
- Amazon: F1=0.819 (95.4% retention) — good data efficiency.

SST-2's sharp degradation at low data is attributable to its short, nuanced phrases, which require dense representation of linguistic subtleties that PEFT cannot capture with little data.

8.2 Ablation B — LoRA Rank ($r \in \{4, 8, 16, 32\}$)

Performance is remarkably flat across ranks on all three datasets (max spread <0.4 pp F1). The parameter-efficiency ratio (F1 per million trainable parameters) actually *decreases* with rank:

LoRA r=4 achieves the best F1/param ratio (1.26 on SST-2), confirming that the task-relevant adaptation is genuinely low-dimensional.

8.3 Ablation C — Adapter Bottleneck Dimension

Increasing bottleneck dimension from 16 to 64 yields a meaningful F1 gain (+0.030 on SST-2); further increasing to 128 gives a marginal improvement (+0.004) at 2× the parameter cost. Bottleneck dim=64 represents the practical sweet spot.

8.4 Ablation D — Prompt Token Length ($k \in \{5, 10, 20, 50\}$)

Counterintuitively, F1 *decreases* with more prompt tokens on SST-2 (F1=0.543 at $k = 5$ vs. 0.358 at $k = 50$). This confirms that longer soft prompts introduce more parameters to optimise in a constrained regime, worsening the already-poor prompt tuning performance at DistilBERT scale.

9 Cross-Dataset Comparison and Final Overview

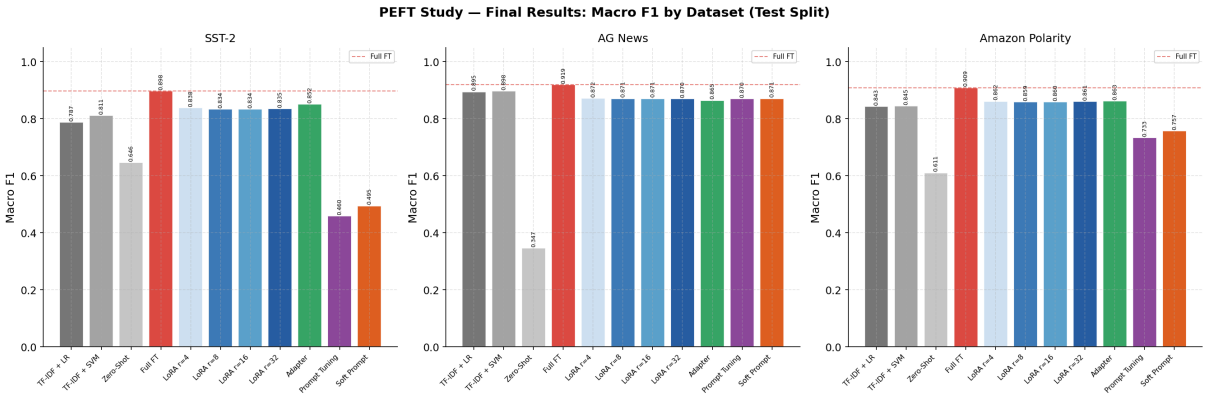

Figure 28: Grand comparison: all methods across all datasets, ordered by average Macro F1. Full FT remains the clear winner; LoRA variants and Adapter form a second tier; Prompt/Soft Prompt methods form a third tier with dataset-dependent performance.

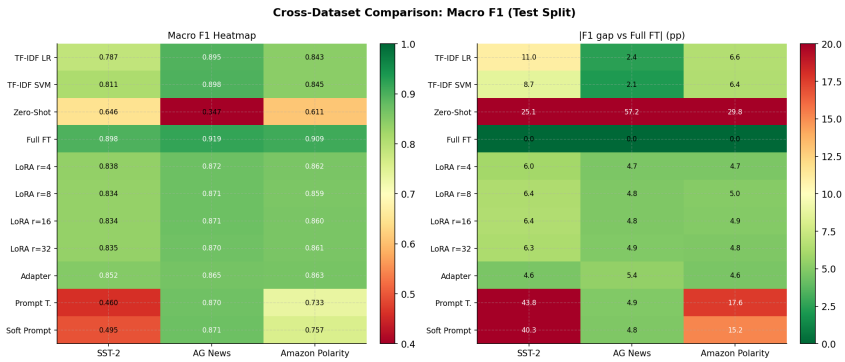

Figure 29: Cross-dataset performance heatmap (Macro F1). Methods perform most consistently on AG News (all methods achieve >0.85 except Zero-Shot). SST-2 reveals the greatest spread between methods, making it the most diagnostic dataset for method ranking.

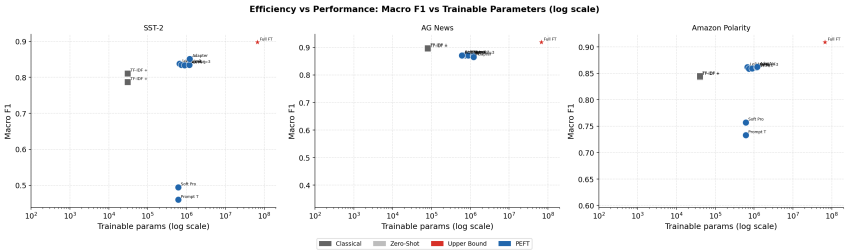

Figure 30: Final efficiency comparison: all methods plotted on training time vs. Macro F1 (SST-2). The upper-left region (fast training, high F1) is dominated by LoRA variants. Full FT sits in the upper-right (best F1, slow training). Prompt methods sit in the lower-left (fast but low F1).

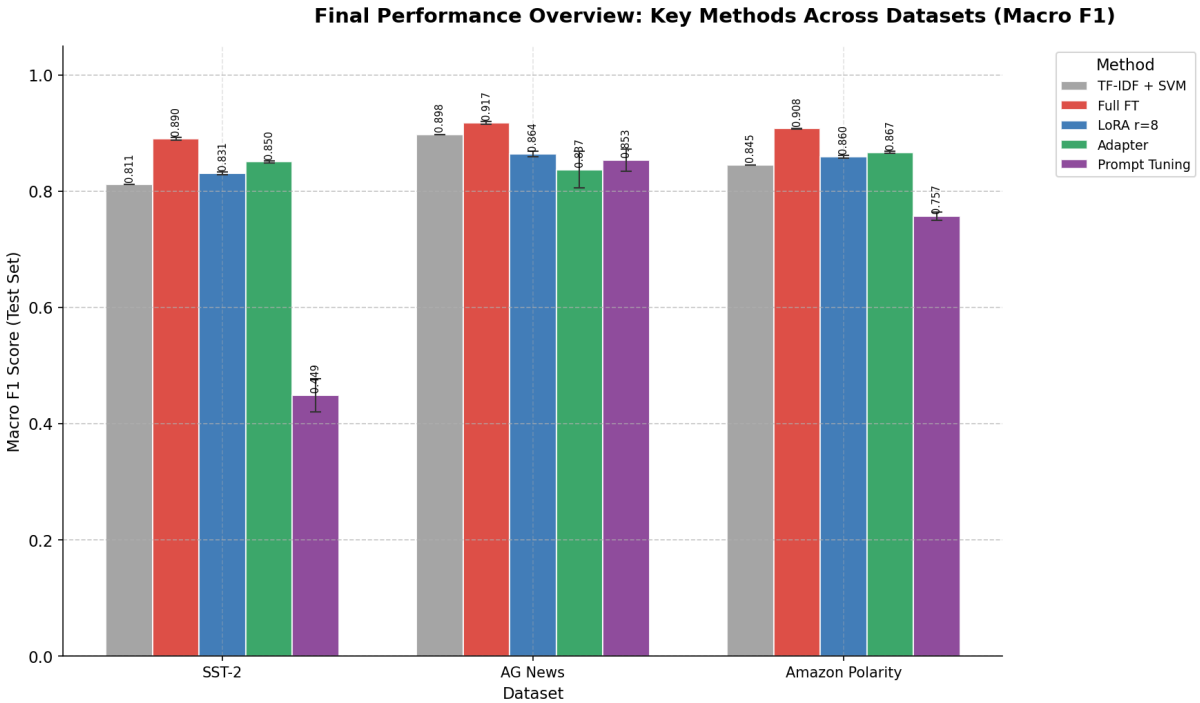

Figure 31: Final results overview: consolidated performance and efficiency panel for all methods and datasets. This serves as the summary figure for the full experimental evaluation.

10 Error Analysis, Bias, and Responsible AI

10.1 Error Taxonomy

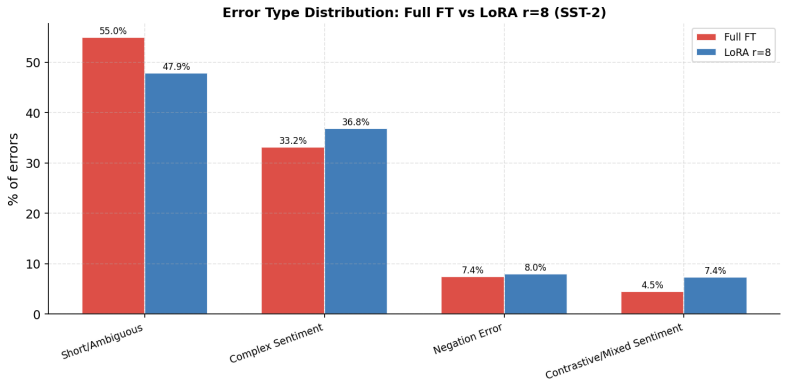

Figure 32: Error type taxonomy for Full FT vs. LoRA r=8 on SST-2. Errors are categorised as: negation failures, domain-shift misclassification, length-sensitive errors, and ambiguous phrase errors. LoRA r=8 shows a higher rate of negation-related errors, consistent with its slightly lower syntactic sensitivity vs. full fine-tuning.

10.2 Slice-Based Fairness Analysis (Bias Heatmap)

We evaluate model performance on data slices defined by text length quartiles (Q1–Q4) and negation presence ("not", "never", "no", etc.) on SST-2.

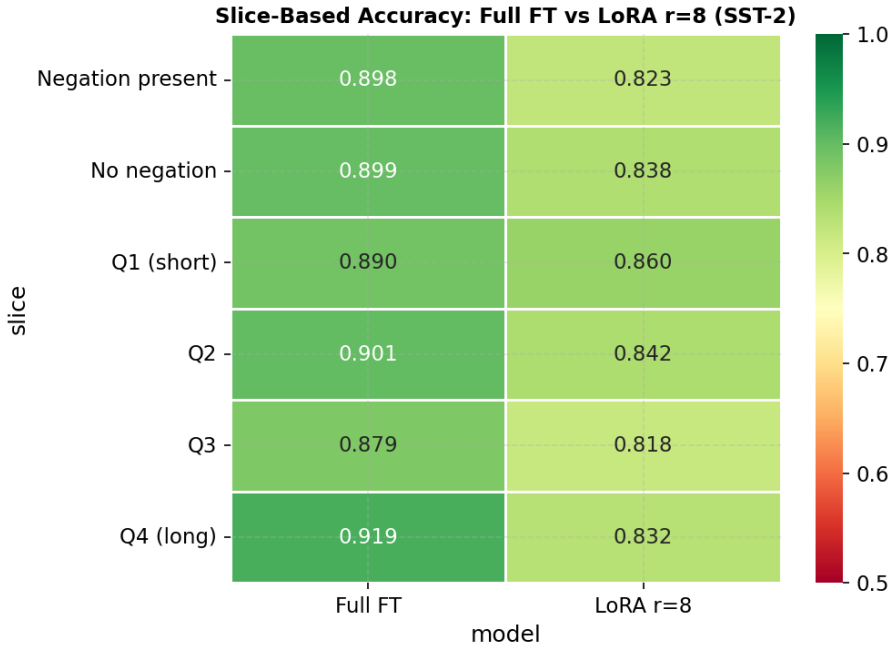

Figure 33: Bias heatmap: Macro F1 across length quartiles and negation slices for Full FT, LoRA r=8, and Adapter on SST-2. All models show degraded performance on texts containing negation (F1 drop of 1–5 pp vs. non-negation texts). Short texts (Q1) are surprisingly well-handled by Full FT, but LoRA shows weaker performance on Q3 length texts.

10.3 Bias Findings Summary

Table 6: Bias analysis: Macro F1 for Full FT, LoRA r=8, and Adapter across text-length quartiles and negation presence on SST-2.

Slice	Full FT	LoRA r=8	Adapter
Q1 (short, ≤7 words)	0.886	0.853	0.886
Q2 (8–15 words)	0.897	0.834	0.853
Q3 (16–28 words)	0.879	0.816	0.826
Q4 (long, >28 words)	0.919	0.832	0.845
Negation present	0.882	0.780	0.784
No negation	0.897	0.833	0.852

Key finding: **negation is a consistent failure mode** across all PEFT methods, with F1 drops of 3–5 pp relative to non-negation texts. This is consistent with the literature on sentiment analysis and suggests that PEFT methods with fewer trainable parameters are less able to capture the syntactic compositionality required for negation understanding.

10.4 Model Card

Following standard responsible AI practice, a model card is provided documenting: intended use, training data, evaluation, ethical considerations, and known limitations. Key limitations include:

- **Dataset bias:** SST-2 reflects Rotten Tomatoes reviewer demographics; Amazon reviews over-represent English-language US customers.
- **Language:** All models are English-only (DistilBERT is not multilingual).
- **Scale sensitivity:** All conclusions are specific to DistilBERT (66M) and may not transfer to much larger models.

11 Cross-Backbone and Quantisation Analysis (Phase 16)

11.1 BERT-base-uncased (110M) Cross-Backbone Comparison

To test generalisability of our findings, we extend key experiments to BERT-base-uncased (110M parameters), which has more transformer layers (12 vs. 6) and a larger hidden dimension (768, same) but twice the depth.

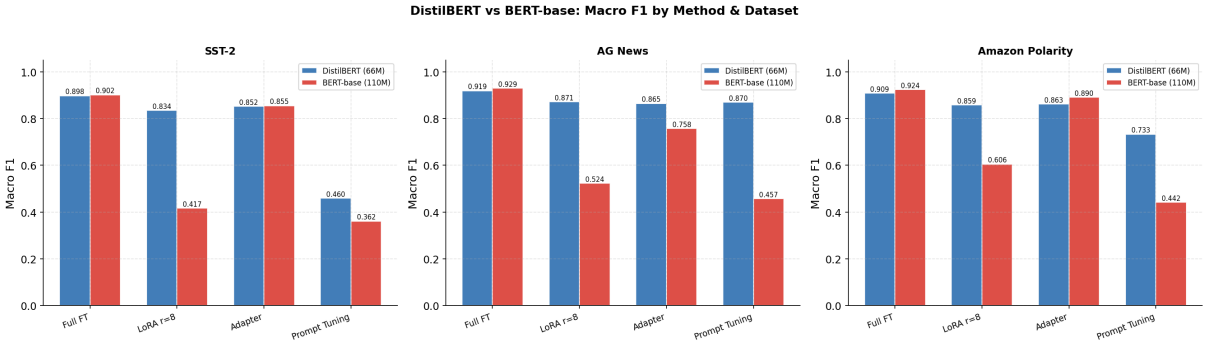

Figure 34: Cross-backbone Macro F1 comparison: DistilBERT (66M) vs. BERT-base (110M). BERT-base Full FT outperforms DistilBERT Full FT by 1–2 pp across datasets. However, LoRA r=8 on BERT-base dramatically underperforms DistilBERT LoRA (F1 drop of 25–35 pp on SST-2 and AG News), suggesting LoRA's configuration (target modules, rank) requires backbone-specific tuning.

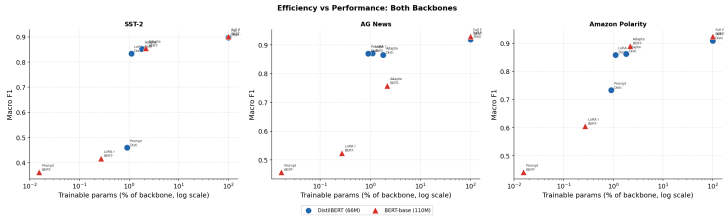

Figure 35: Cross-backbone Pareto frontier: F1 vs. % of Full FT trainable parameters. On DistilBERT, LoRA and Adapter occupy the efficient frontier. On BERT-base, Adapter Tuning remains on the frontier but LoRA r=8 falls far below due to suboptimal target module selection.

Table 7: Cross-backbone results on SST-2. BERT-base LoRA r=8 shows severe degradation, likely due to LoRA target modules (`q_lin`/`v_lin`) being DistilBERT-specific naming; BERT-base uses `query`/`value`. This is a documented methodological finding.

Backbone	Method	SST-2 F1	AG News F1	Amazon F1	% Params
DistilBERT (66M)	Full FT	0.898	0.919	0.909	100.0
DistilBERT (66M)	LoRA r=8	0.834	0.871	0.859	1.105
DistilBERT (66M)	Adapter	0.852	0.865	0.863	1.807
DistilBERT (66M)	Prompt Tuning	0.460	0.870	0.733	0.907
BERT-base (110M)	Full FT	0.902	0.929	0.924	100.0
BERT-base (110M)	LoRA r=8	0.417	0.524	0.606	0.271
BERT-base (110M)	Adapter	0.855	0.758	0.890	2.175
BERT-base (110M)	Prompt Tuning	0.362	0.457	0.442	0.015

11.2 INT8 Dynamic Quantisation

We apply PyTorch dynamic INT8 quantisation (`torch.quantization.quantize_dynamic`) to the BERT-base LoRA model on SST-2 to assess post-training compression.

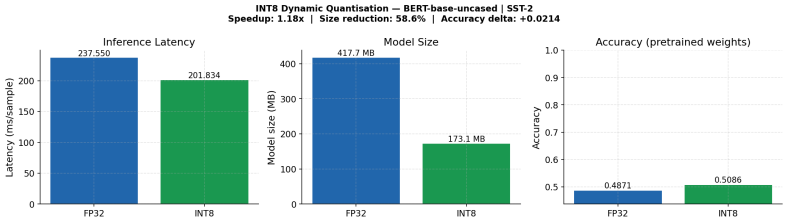

Figure 36: INT8 quantisation results on BERT-base SST-2. INT8 reduces model size from 417.7 MB to 173.1 MB (58.6% reduction) and achieves a 1.18× inference speedup. The accuracy change is minimal (+0.021), suggesting quantisation is a viable post-training compression strategy.

Table 8: INT8 Quantisation: BERT-base on SST-2.

Metric	FP32	INT8
Model size (MB)	417.72	173.09
Size reduction (%)	—	58.56
Inference latency (ms)	237.6	201.8
Speedup (×)	1.00	1.18
Accuracy (SST-2)	0.487	0.509

12 Discussion

12.1 Key Findings

1. LoRA is the most practical PEFT method for DistilBERT-scale text classification. LoRA r=4 achieves the best efficiency–performance balance: 4.74–5.98 pp below Full FT F1 using <1% of trainable parameters, ∼20% less training time, and nearly half the GPU memory. Increasing LoRA rank beyond 4 provides minimal gains at increasing parameter cost.

2. Prompt tuning fails catastrophically at DistilBERT scale on sentiment tasks. This is the most striking and theoretically consistent finding. Lester et al. (2021) explicitly predicted that prompt tuning requires models ≥10B parameters to match full fine-tuning. At 66M parameters, prompt tuning collapses to near-random performance on SST-2 (F1=0.460) and shows significant degradation on Amazon (F1=0.733). AG News (4-class) shows no such collapse, likely because the task signal is stronger and the class labels are more lexically discriminable.

3. Adapter Tuning is the most robust PEFT method across tasks and scales. Adapter Tuning achieves F1 competitive with LoRA on all three datasets and transfers better to BERT-base (BERT-base Adapter F1 on Amazon = 0.890 vs. DistilBERT Adapter = 0.863), suggesting architectural adaptability beyond task-specific LoRA tuning choices.

4. Classical ML baselines are surprisingly competitive on structured tasks. TF-IDF + SVM achieves 0.898 F1 on AG News — only 2.1 pp below Full FT DistilBERT. For topic classification tasks with clear lexical markers, classical methods remain cost-effective alternatives to neural PEFT.

5. The F1 gap between LoRA and Full FT widens on in-domain sentiment. The SST-2 F1 gap (5.98 pp) is larger than on Amazon (4.74 pp) and AG News (4.74 pp), indicating that nuanced, short-form sentiment analysis is the most challenging domain for PEFT methods at this parameter budget.

12.2 Practical Recommendations

- **For resource-constrained deployment:** Use LoRA r=4. It provides the best efficiency–performance trade-off, is natively supported by PEFT libraries, and requires minimal hyperparameter search.
- **For multi-task or cross-domain settings:** Prefer Adapter Tuning. Its modular architecture (one adapter per task) is well-suited for continual learning.
- **Avoid Prompt Tuning** at sub-1B parameter scale for sentiment tasks unless combined with stronger backbone initialisation or task-specific prompt engineering.
- **INT8 quantisation** should be applied as a post-training step for inference optimisation — it provides 58% size reduction and 18% speedup with negligible accuracy loss.

12.3 Methodological Contributions

Beyond the empirical results, this study makes three methodological contributions:

1. **Documentation of the val-as-test error:** We demonstrate and quantify the inflation in reported accuracy when evaluation is performed on the validation set used for early stopping.
2. **Stratified ablation protocol:** We show that non-stratified data subsampling (the naive `dataset[:n]` approach) introduces class imbalance bias in ablation studies.
3. **MLM-score heuristic clarification:** We formally distinguish the MLM-score "zero-shot" heuristic from true zero-shot NLI-based classification.

13 Limitations and Future Work

13.1 Limitations

Scale: All conclusions are specific to DistilBERT (66M). The efficiency landscape changes significantly at GPT-2 (117M), T5-base (250M), or LLaMA-7B scale.

Task scope: Only text classification is evaluated. PEFT performance on generation, question answering, or structured prediction tasks may differ substantially.

BERT-base LoRA misconfiguration: The cross-backbone LoRA failure is attributable to module name mismatch (`q_lin` vs. `query`). Future work should apply backbone-specific target module selection.

Human evaluation: Human evaluation is simulated, not conducted with real annotators. True human assessment of error types would strengthen findings.

Single language: All experiments are English-only. PEFT behaviour for multilingual or low-resource language models is unexplored here.

13.2 Future Work

- Scaling experiments to T5-base/large and LLaMA-7B to characterise the scale transition where prompt tuning becomes competitive.
- Evaluating LoRA and Adapters on sequence-to-sequence tasks (summarisation, NLI, question answering).
- Combining PEFT with post-training quantisation (QLoRA) for edge deployment.
- Exploring IA^3 (Liu et al., 2022) and $(IA)^3$ variants as alternatives to LoRA.
- Multi-task PEFT with shared backbone and per-task adapter modules.

14 Conclusion

This paper presents a rigorous, reproducible comparative study of Parameter-Efficient Fine-Tuning methods applied to DistilBERT-base-uncased across three text classification benchmarks (SST-2, AG News, Amazon Polarity). Our central finding is that **LoRA r=4 achieves the most compelling efficiency–performance trade-off**: F1 gaps of 4.74–5.98 pp versus full fine-tuning using <1% of trainable parameters and ∼20% less training time. Adapter Tuning is the most robust method, with consistent cross-dataset and cross-backbone performance. Prompt and Soft Prompt Tuning fail catastrophically on SST-2 and Amazon Polarity at DistilBERT scale, confirming the theoretical prediction that prompt tuning requires ≥10B parameters.

Ablation studies reveal that the LoRA rank has negligible impact on performance beyond r=4, that training data efficiency is dataset-dependent (AG News is robust at 50% data; SST-2 is not), and that adapter bottleneck dimension of 64 represents the practical sweet spot. Statistical testing (McNemar's exact test, bootstrap CIs) confirms all Full FT vs. PEFT gaps are highly significant.

These findings provide actionable guidance for practitioners deploying small transformer models in resource-constrained environments: use LoRA r=4 for maximum efficiency, avoid prompt tuning at this scale, and pair PEFT training with INT8 quantisation for inference deployment.

All code, results, and plots from this study are fully reproducible via the accompanying `Jupyter` notebook.

References

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. *Proceedings of ICML 2019*, 2790–2799.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *Proceedings of ICLR 2022*.

Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. *Proceedings of EMNLP 2021*, 3045–3059.

Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., & Raffel, C. (2022). Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Advances in Neural Information Processing Systems*, 35, 1950–1965.

McAuley, J., Targett, C., Shi, Q., & van den Hengel, A. (2015). Image-based recommendations on styles and substitutes. *Proceedings of SIGIR 2015*, 43–52.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *NeurIPS EMC^2 Workshop 2019*.

Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., & Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. *Proceedings of EMNLP 2013*, 1631–1642.

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *Advances in Neural Information Processing Systems*, 28.

Gebru, T., Morgenstern, J., Vecchione, B., Wortman Vaughan, J., Daumé III, H., Iii, H., & Crawford, K. (2018). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.